Recruitment Management System

Technology Stack & Implementation Guide

Backend Node.js · TypeScript · Express · Prisma · PostgreSQL

Frontend Next.js · TypeScript · MUI · Redux Toolkit + RTK Query

Realtime Socket.IO · **Cache/Queue** Redis

Why each choice · How the pieces fit together

The mistakes this stack invites, and how to avoid them

38 stack-specific interview questions with answers

Contents

1. Verdict on Your Stack
1.1 What is right · 1.2 Two adjustments · 1.3 The one-sentence justification

2. System Architecture

3. Backend — Node.js, TypeScript, Express
3.1 Layering · 3.2 Folder structure · 3.3 Error handling · 3.4 Validation

4. Prisma in Depth
4.1 Schema modelling · 4.2 What Prisma cannot express · 4.3 Transactions and locking
4.4 The N+1 problem · 4.5 Money and Decimal · 4.6 Migrations · 4.7 Connection pooling

5. Authentication & Security

6. Real-Time Chat with Socket.IO

7. Frontend — Next.js App Router
7.1 Rendering strategy per page · 7.2 Server vs Client Components

8. Material UI
8.1 Why MUI · 8.2 The App Router setup gotcha · 8.3 Theming · 8.4 Performance

9. Redux Toolkit & RTK Query
9.1 Client state vs server state · 9.2 Slice design · 9.3 RTK Query patterns

10. Project Structure — Both Repositories

11. Testing Strategy

12. Deployment

13. Interview Question Bank (38 Questions)

14. Trade-offs & What You Would Change

15. Build Order — What to Do First

1. Verdict on Your Stack

1.1 What is right about it

This is a coherent, current, and defensible stack. Every choice fits the problem, and three of them fit it unusually well.

Choice	Verdict	Reasoning
TypeScript everywhere	STRONG	One language across both tiers, and — critically — Prisma generates types directly from your schema, so a column rename surfaces as a compile error in your React component. For a 41-table schema this is the single highest-leverage decision on the list.
Prisma	STRONG	Type-safe queries, readable relation loading, and a genuinely good migration workflow. Its weakness is complex SQL — which matters here, and §4.2 covers exactly where you must drop to raw SQL.
PostgreSQL	STRONG	Required by your design. Partial indexes, CHECK constraints and transactional guarantees are load-bearing in this schema.
Next.js	STRONG	Your app has a public SEO-critical half (job listings, company pages) and a private interactive half (dashboards, pipeline, chat). Next serves both from one codebase.
MUI	SOLID	DataGrid, date pickers, dialogs and form controls out of the box — you will use all of them. Strong signal in enterprise hiring, especially in India. Costs bundle size and a somewhat generic look unless themed.
Redux Toolkit	SOLID	Correct for auth, UI and filter state. The risk is using it for server data, which §9 addresses.
Node.js + Express	SOLID	Fine, provided you impose structure yourself. Express gives you none.

1.2 Two adjustments I would make

Adjustment 1 — Use RTK Query for server data, not hand-written thunks.

RTK Query ships *inside* Redux Toolkit, so this adds no dependency. The most common failure with this exact stack is writing a `fetchJobs` thunk plus `loading` / `error` / `data` fields in every slice — roughly four hundred lines of boilerplate that reimplements caching, deduplication and invalidation badly. RTK Query gives you all of that declaratively. Redux then holds only genuine client state: the authenticated user, UI toggles, filter selections, unsent chat drafts.

Getting this boundary right is also an interview differentiator. Most candidates cannot articulate the difference between client state and server state; being able to is a strong signal.

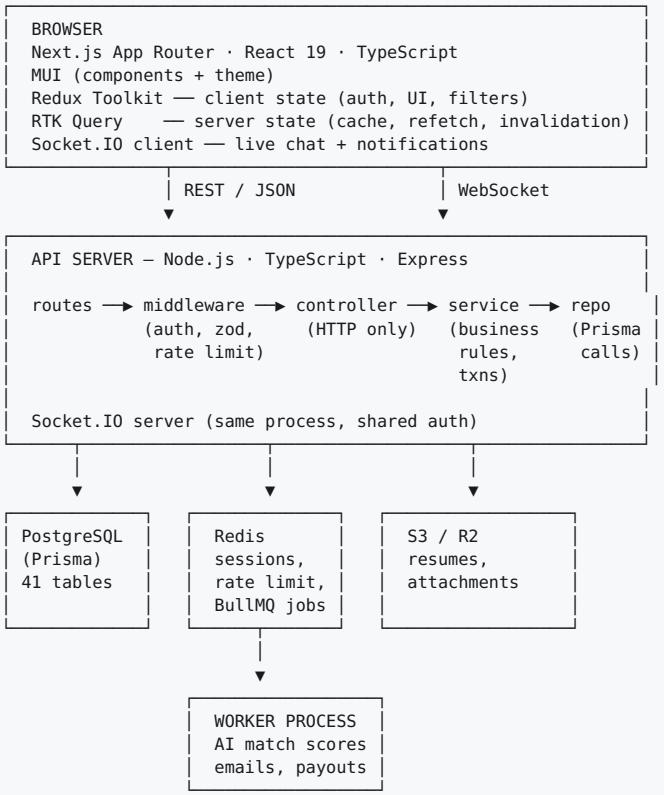
Adjustment 2 — "Node.js" needs a framework decision.

Nobody builds on bare `http`. Your realistic options are **Express** (minimal, ubiquitous, you supply all structure) or **NestJS** (opinionated modules, dependency injection, decorators, structure enforced for you). This guide is written for Express because it is the more common answer and keeps the mental model small. If you would rather have the architecture imposed on you rather than maintained by discipline, NestJS is the better choice and everything in §3 and §4 still applies — only the wiring changes.

1.3 The one-sentence justification to have ready

"TypeScript end to end with Prisma means my database schema generates types that flow all the way into React components, so a schema change becomes a compile error rather than a runtime bug — which matters a lot on a forty-one table schema. Postgres was non-negotiable because the payment flow needs real transactions and my integrity rules rely on partial indexes and CHECK constraints. Next.js because job listings need server rendering for SEO while the dashboards need to be interactive, and it does both. MUI for component velocity, and Redux Toolkit with RTK Query so client state and server state stay clearly separated."

2. System Architecture



Three deployable processes, not three microservices: an API server, a worker, and the Next.js frontend. They share one database and one codebase for types. That distinction — separate *processes*, single *system* — is worth stating precisely in an interview, because it shows you understand why you did not reach for microservices.

3. Backend — Node.js, TypeScript, Express

3.1 The layering rule

Express gives you no structure, so you must impose it. Three layers, with one responsibility each and a hard rule about what may call what.

Layer	Responsibility	Must never
Controller	Parse the request, validate shape, call one service, map the result to a status code	Contain business rules or touch Prisma
Service	Business rules, authorization decisions, transaction boundaries, orchestration across repositories	Know about `req`, `res`, or HTTP status codes
Repository	All Prisma calls; returns domain objects	Contain business logic or decide what is allowed

> **The test for whether your layering is real:** could you add a CLI command or a queue consumer that reuses the service without touching the controller? If the service reads `req.user` or throws HTTP errors, the answer is no, and your layers are decorative.

3.2 Backend folder structure

```
backend/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── src/
│   ├── config/            env parsing (zod-validated), constants
│   ├── modules/
│   │   ├── auth/          auth.controller.ts · auth.service.ts · auth.routes.ts
│   │   ├── users/
│   │   ├── jobs/
│   │   ├── applications/
│   │   ├── matching/
│   │   ├── proposals/
│   │   ├── contracts/
│   │   ├── milestones/
│   │   ├── payments/      escrow, wallet, ledger — the critical module
│   │   ├── chat/
│   │   ├── disputes/
│   │   └── admin/
│   ├── middleware/        authenticate, authorize, validate, rateLimit, errorHandler
│   ├── shared/
│   │   ├── prisma.ts      singleton PrismaClient
│   │   ├── redis.ts
│   │   ├── errors.ts      AppError hierarchy
│   │   ├── events.ts      domain event emitter
│   │   └── types.ts
│   ├── sockets/           Socket.IO namespaces + handlers
│   ├── jobs/              BullMQ queues and workers
│   ├── app.ts             Express wiring
│   └── server.ts          HTTP + Socket.IO bootstrap
└── tests/
```

Group by **feature**, not by technical type. A `controllers/` folder holding thirty unrelated controllers means every change touches four distant directories. Feature folders keep a change local, and they make the module boundaries from your architecture visible in the filesystem.

3.3 Error handling that does not leak

```
// shared/errors.ts
export class AppError extends Error {
  constructor(
    public readonly statusCode: number,
    message: string,
    public readonly code: string,
    public readonly isOperational = true,
  ) { super(message); }
}

export class NotFoundError extends AppError {
  constructor(resource: string) { super(404, `${resource} not found`, 'NOT_FOUND'); }
}
export class ForbiddenError extends AppError {
  constructor(msg = 'Not permitted') { super(403, msg, 'FORBIDDEN'); }
}
export class ConflictError extends AppError {
  constructor(msg: string) { super(409, msg, 'CONFLICT'); }
}

// middleware/errorHandler.ts — must be registered LAST
export const errorHandler = (
  err: Error, _req: Request, res: Response, _next: NextFunction,
) => {
  if (err instanceof AppError && err.isOperational) {
    return res.status(err.statusCode).json({ error: err.code, message: err.message });
  }
  // Prisma's unique-constraint violation → 409, not 500
  if (err instanceof Prisma.PrismaClientKnownRequestError && err.code === 'P2002') {
    return res.status(409).json({ error: 'DUPLICATE', message: 'Already exists' });
  }
  logger.error({ err }, 'Unhandled error');
  res.status(500).json({ error: 'INTERNAL', message: 'Something went wrong' });
};
```

Worth saying in an interview: mapping Prisma's `P2002` to a 409 means your database constraints become correct API responses automatically. When a candidate applies twice, the partial unique index rejects it and the client receives a clean "already applied" — the rule is enforced once, in the database, and surfaces properly without duplicated checks in the service layer.

3.4 Validation with Zod

Validate at the boundary, then trust the data inward. Zod is the natural fit because it infers TypeScript types from the schema, so you define the shape once.

```
// jobs/job.schema.ts
export const createJobSchema = z.object({
  title:       z.string().min(5).max(200),
  description: z.string().min(50),
  jobType:     z.enum(['FULL_TIME', 'FREELANCE']),
  salaryMin:   z.number().positive().optional(),
  salaryMax:   z.number().positive().optional(),
  budget:      z.number().positive().optional(),
  skillIds:    z.array(z.string().uuid()).min(1).max(20),
}).refine(
  (d) => d.jobType === 'FULL_TIME'
    ? d.budget === undefined
    : d.salaryMin === undefined && d.budget !== undefined,
  { message: 'Full-time jobs take a salary range; freelance jobs take a budget' },
).refine(
  (d) => !d.salaryMin || !d.salaryMax || d.salaryMin <= d.salaryMax,
  { message: 'salaryMin must not exceed salaryMax' },
);

export type CreateJobInput = z.infer<typeof createJobSchema>;
```

Notice this mirrors the CHECK constraints from your database design. That duplication is intentional and worth defending: the database constraint is the guarantee, the Zod schema is the *good error message*. Users should get "salary range is backwards", not a raw constraint violation.

4. Prisma in Depth

4.1 Modelling your schema

```
model Contract {
  id                String   @id @default(uuid()) @db.Uuid
  jobId             String   @map("job_id") @db.Uuid
  proposalId        String   @unique @map("proposal_id") @db.Uuid
  clientId          String   @map("client_id") @db.Uuid
  freelancerId      String   @map("freelancer_id") @db.Uuid
  agreedAmount      Decimal  @map("agreed_amount") @db.Decimal(12, 2)
  currency          String   @default("INR") @db.VarChar(3)
  platformFeePercent Decimal  @map("platform_fee_percent") @db.Decimal(5, 2)
  status            ContractStatus @default(PENDING_FUNDING)
  startedAt         DateTime? @map("started_at")
  createdAt         DateTime @default(now()) @map("created_at")

  job        Job              @relation(fields: [jobId], references: [id])
  proposal   Proposal         @relation(fields: [proposalId], references: [id])
  client     RecruiterProfile @relation(fields: [clientId], references: [id])
  freelancer CandidateProfile @relation(fields: [freelancerId], references: [id])
  milestones Milestone[]
  reviews    Review[]
  chatRoom   ChatRoom?

  @@index([freelancerId, status])
  @@index([clientId, status])
  @@map("contracts")
}

enum ContractStatus {
  PENDING_FUNDING
  ACTIVE
  COMPLETED
  CANCELLED
  DISPUTED
}
```

`@map` and `@@map` let you keep `snake_case` in Postgres — the SQL convention — while writing `camelCase` in TypeScript. Set this up from the first model; retrofitting it later means rewriting every query.

4.2 What Prisma cannot express — and what to do about it

> **This is the most important section in the guide for your project, and a question you should expect.** Your schema depends on partial unique indexes and CHECK constraints. Prisma's schema language supports neither. If you only use `prisma migrate`, those rules silently never exist in your database.

The solution is to generate an empty migration and write the SQL by hand. Prisma tracks it in migration history exactly like a generated one.

```
# Generate an empty migration file
npx prisma migrate dev --create-only --name add_integrity_constraints

# Then edit prisma/migrations/<timestamp>_add_integrity_constraints/migration.sql
```

```
-- Partial unique index: one live application per candidate per job
CREATE UNIQUE INDEX "uq_applications_active"
  ON "applications" ("job_id", "candidate_id")
  WHERE "status" NOT IN ('WITHDRAWN', 'REJECTED');

-- CHECK: AI weights must total 100
ALTER TABLE "ai_weightages"
  ADD CONSTRAINT "ck_ai_weights_sum"
  CHECK ("skill_weight" + "experience_weight"
       + "education_weight" + "project_weight" = 100);

-- CHECK: a chat room has exactly one parent
ALTER TABLE "chat_rooms"
  ADD CONSTRAINT "ck_chat_room_single_parent"
  CHECK (num_nonnulls("contract_id", "application_id") = 1);

-- CHECK: nobody reviews themselves
ALTER TABLE "reviews"
  ADD CONSTRAINT "ck_review_not_self"
  CHECK ("reviewer_id" <> "reviewee_id");
```

Run `npx prisma migrate dev` afterwards to apply it. Because Prisma introspects state rather than reverse-engineering these, they persist safely — but never run `prisma db push` on a database holding these constraints, as it can drop what the schema file does not know about.

4.3 Transactions and row locking — the milestone release

This is the code an interviewer is most likely to ask you to walk through, because it is where correctness actually matters.

```
async function releaseMilestone(milestoneId: string, approverId: string) {
  return prisma.$transaction(async (tx) => {

    // 1. Lock the wallet row. Prisma has no FOR UPDATE API,
    //    so this must be raw SQL — inside the same transaction client.
    const [wallet] = await tx.$queryRaw<Wallet[]>`
      SELECT * FROM wallets WHERE user_id = ${freelancerUserId} FOR UPDATE
    `;

    // 2. Guarded state transition. If another request already approved this,
    //    count is 0 and we abort instead of paying twice.
    const updated = await tx.milestone.updateMany({
      where: { id: milestoneId, status: 'SUBMITTED' },
      data:  { status: 'APPROVED', approvedAt: new Date() },
    });
    if (updated.count === 0) {
      throw new ConflictError('Milestone is not awaiting approval');
    }

    const milestone = await tx.milestone.findUniqueOrThrow({
      where: { id: milestoneId },
      include: { contract: true, escrow: true },
    });

    // 3. Release escrow
    await tx.escrow.update({
      where: { milestoneId },
      data:  { status: 'RELEASED', releasedAt: new Date() },
    });

    // 4. Compute the split using Decimal, never JS numbers
    const feePct = milestone.contract.platformFeePercent;
    const fee    = milestone.amount.mul(feePct).div(100);
    const net    = milestone.amount.sub(fee);

    await tx.platformFee.create({
      data: { contractId: milestone.contractId, feeType: 'FREELANCE_COMMISSION',
              amount: fee, percentage: feePct },
    });

    // 5. Credit wallet and write the ledger entry together
    const newBalance = wallet.availableBalance.add(net);
    await tx.wallet.update({
      where: { id: wallet.id },
      data:  { availableBalance: newBalance },
    });
    await tx.transaction.create({
      data: { walletId: wallet.id, direction: 'CREDIT', amount: net,
              balanceAfter: newBalance, referenceType: 'ESCROW',
              referenceId: milestone.escrow!.id,
              description: 'Milestone released' },
    });

    return { milestoneId, net, fee };
  }, {
    isolationLevel: 'ReadCommitted',
    timeout: 10_000,
  });
}
```

Four things to point out when explaining this code:

1. Everything uses `tx`, not `prisma`. Using the global client inside a transaction callback opens a *separate* connection outside the transaction — a subtle bug that breaks atomicity while appearing to work.

2. The `FOR UPDATE` lock must be raw SQL because Prisma exposes no API for it. This is a legitimate, well-known limitation, and knowing it is a strong signal.

3. `updateMany` with a status guard makes the transition idempotent at the database level. A double-clicked approve button cannot pay twice.

4. Money arithmetic uses `Decimal` methods (`.mul()`, `.sub()`), never JavaScript `*` and `-`.

4.4 The N+1 problem

The single most common Prisma performance bug, and a guaranteed interview question.

```
// ✗ N+1: one query for the list, then one per row = 51 queries
const applications = await prisma.application.findMany({ where: { jobId } });
for (const app of applications) {
  const candidate = await prisma.candidateProfile.findUnique({
    where: { id: app.candidateId },
  });
}

// ✓ One query with a join
const applications = await prisma.application.findMany({
  where: { jobId },
  include: {
    candidate: {
      include: { user: { select: { fullName: true, email: true } } },
    },
    currentStage: true,
  },
  orderBy: { appliedAt: 'desc' },
  take: 50,
});
```

Use `select` rather than `include` when you need only a few fields — `include` fetches every column of the relation, which on a wide table means moving data you discard. In development, enable query logging (`new PrismaClient({ log: ['query'] })`) and watch the count; if one page load emits fifty queries, you have found an N+1.

4.5 Money and the Decimal type

```
// ✗ Wrong — floating point cannot represent 0.1 exactly
const fee = amount * 0.15;        // 1234.56 * 0.15 → 185.18400000000003

// ✓ Right — Prisma.Decimal, backed by decimal.js
import { Prisma } from '@prisma/client';
const fee = amount.mul(new Prisma.Decimal('0.15'));
const net = amount.sub(fee);

// Serialising to JSON: Decimal is an object, not a number
res.json({ amount: amount.toFixed(2) });   // "1234.56"
```

> Prisma returns `Decimal` objects, which `JSON.stringify` renders as `{"s":1,"e":3,"d":[...]}` unless you convert. Decide the boundary rule early: convert to a fixed-precision string in the controller, and have the frontend treat money as a string until display. Doing arithmetic on money in the browser is how rounding bugs reach users.

4.6 Migrations

Command	Use
`prisma migrate dev`	Development. Generates SQL from schema changes, applies it, regenerates the client.
`prisma migrate dev --create-only`	Generate the file without applying — required when you need to hand-edit SQL for CHECK constraints and partial indexes.
`prisma migrate deploy`	**Production.** Applies pending migrations only. Never generates, never prompts, never resets.
`prisma db push`	Prototyping only. Skips migration history — never run against production or any database holding hand-written constraints.
`prisma generate`	Regenerates the typed client. Must run after every schema change and in your CI build step.

4.7 Connection pooling — the deployment trap

> Each `PrismaClient` instance opens its own connection pool. Two consequences that bite people in production:
>
> **Always use a singleton.** In development, Next.js and `ts-node-dev` hot reloading creates a new client on every reload until Postgres refuses connections. The standard fix is to cache the instance on `globalThis` outside production.
>
> **Serverless needs a pooler.** If you deploy the API to a serverless platform, every cold start opens a pool and Postgres' default limit of ~100 connections is exhausted quickly. Use PgBouncer in transaction mode with `?pgbouncer=true` in the connection string, or Prisma Accelerate. Deploying the API as a long-running process on Railway, Render or Fly.io sidesteps the problem entirely — which is what I would recommend for this project.

```
// shared/prisma.ts
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const prisma = globalForPrisma.prisma ?? new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['query', 'warn', 'error'] : ['error'],
});

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

5. Authentication & Security

5.1 Token strategy

Use a short-lived access token plus a long-lived refresh token, with the refresh token in an `httpOnly` cookie.

Token	Lifetime	Stored	Why
Access (JWT)	15 minutes	Memory / Redux	Short life limits the damage if it leaks; never in `localStorage`, which any XSS can read
Refresh	7 days	`httpOnly`, `Secure`, `SameSite=Strict` cookie	JavaScript cannot read it, so XSS cannot steal it; rotated on every use

> **The question you will be asked:** "*Why not store the JWT in localStorage?*" Because any cross-site scripting vulnerability — including one in a third-party npm package you did not write — can read `localStorage` and exfiltrate the token. An `httpOnly` cookie is unreadable from JavaScript. The trade-off is that cookies are vulnerable to CSRF, which you mitigate with `SameSite=Strict` and a CSRF token on state-changing requests. Naming both sides of that trade is the complete answer.

5.2 Authorization must be resource-level, not just role-level

```
// ✗ Insufficient — any recruiter can now read any job's applications
router.get('/jobs/:id/applications', authenticate, authorize('RECRUITER'), handler);

// ✓ Ownership checked in the service layer
async function getApplicationsForJob(jobId: string, userId: string) {
  const job = await prisma.job.findUnique({
    where: { id: jobId },
    select: { id: true, recruiter: { select: { userId: true } } },
  });
  if (!job) throw new NotFoundError('Job');
  if (job.recruiter.userId !== userId) throw new ForbiddenError();

  return prisma.application.findMany({ where: { jobId }, include: { candidate: true } });
}
```

This class of bug — **Insecure Direct Object Reference** — is the most common real vulnerability in marketplace applications. Role checks answer "is this person a recruiter"; they do not answer "is this *their* job". Being able to name IDOR is a strong security signal.

5.3 Security checklist

Concern	Measure
Password storage	`bcrypt` with cost 12, or `argon2id`. Never MD5 or bare SHA-256 — speed is the wrong property.
SQL injection	Prisma parameterises everything. Raw queries must use the tagged-template form `$queryRaw`...${v}``, never `$queryRawUnsafe` with string concatenation.
Rate limiting	`express-rate-limit` with a Redis store. Tight on login and proposal submission, which are the abuse targets.
Headers	`helmet()` for CSP, HSTS, X-Frame-Options.
File uploads	Validate MIME *and* magic bytes, cap size, generate presigned S3 URLs so files never transit your API server.
Secrets	Zod-validated env parsing at boot, so a missing `JWT_SECRET` crashes on startup rather than at 3am.

6. Real-Time Chat with Socket.IO

6.1 Authenticating the socket

```
io.use(async (socket, next) => {
  try {
    const token = socket.handshake.auth.token;
    const payload = jwt.verify(token, env.JWT_SECRET) as JwtPayload;
    socket.data.userId = payload.sub;
    next();
  } catch {
    next(new Error('UNAUTHORIZED'));
  }
});

io.on('connection', (socket) => {
  socket.on('room:join', async (roomId: string) => {
    // Membership is checked against chat_participants — never trust the client
    const member = await prisma.chatParticipant.findFirst({
      where: { roomId, userId: socket.data.userId },
    });
    if (!member) return socket.emit('error', { code: 'FORBIDDEN' });
    socket.join(`room:${roomId}`);
  });

  socket.on('message:send', async ({ roomId, body }) => {
    const member = await prisma.chatParticipant.findFirst({
      where: { roomId, userId: socket.data.userId },
    });
    if (!member) return;

    // Persist FIRST, then broadcast. The socket is transport, not storage.
    const message = await prisma.message.create({
      data: { roomId, senderId: socket.data.userId, body },
    });
    io.to(`room:${roomId}`).emit('message:new', message);
  });
});
```

Two points worth making unprompted. First, authorization is re-checked on every socket event, not only at connection — a client can emit any event name with any payload, so the socket connection is not a trust boundary. Second, messages are written to Postgres before being broadcast. Your dispute-resolution flow depends on message history existing; a message that only ever lived in a WebSocket frame is evidence that does not exist.

Deployment constraint. Socket.IO needs a persistent connection, so it cannot run on Vercel serverless functions. Run it in your Express process on Railway, Render or Fly.io. If you later run more than one API instance, add the Redis adapter (`@socket.io/redis-adapter`) so a broadcast reaches clients connected to other instances — otherwise two users in the same chat may be on different servers and never see each other's messages.

7. Frontend — Next.js App Router

7.1 Choosing a rendering strategy per page

The reason Next.js is the right choice here is that your pages genuinely differ. Being able to justify the strategy page by page is a strong interview answer.

Page	Strategy	Why
Landing, company pages	Static (SSG)	Rarely changes; serve from CDN
Job listing / detail	Server-rendered, revalidated	**SEO is the point.** A job board Google cannot index has a product problem, not a technical one
Candidate dashboard	Client-side	Private, user-specific, no SEO value
Pipeline board	Client-side	Drag-and-drop, highly interactive
Chat	Client-side	WebSocket-driven
Admin panels	Client-side	Private, data-grid heavy

7.2 Server Components and the boundary

In the App Router every component is a Server Component unless marked `'use client'`. Server Components run only on the server, ship no JavaScript to the browser, and can query data directly — but they cannot use hooks, state, effects, or event handlers.

```
// app/jobs/[id]/page.tsx — Server Component, no 'use client'
export default async function JobPage({ params }: { params: { id: string } }) {
  const job = await fetch(`${API}/jobs/${params.id}`, {
    next: { revalidate: 300 },
  }).then(r => r.json());

  return (
    <>
      <JobHeader job={job} />            {/* server-rendered, SEO-visible */}
      <ApplyButton jobId={job.id} />     {/* 'use client' — needs onClick */}
    </>
  );
}

export async function generateMetadata({ params }) {
  const job = await getJob(params.id);
  return { title: `${job.title} at ${job.company.name}`,
           description: job.description.slice(0, 160) };
}
```

The friction you will hit: Redux and MUI are both client-side by nature. Anything using `useSelector`, `useState` or MUI's `sx` with a theme must be a Client Component. The practical pattern is to keep Server Components at the page level for data fetching and SEO metadata, then drop into a Client Component boundary as soon as you need interactivity. Do not fight this — put `'use client'` at the top of your dashboard layouts and treat that whole subtree as a normal React SPA.

8. Material UI

8.1 Why MUI is a reasonable choice here

Your product needs data grids with sorting and pagination, date pickers, modals, form controls, tabs, menus, and a notification system. MUI ships all of it, accessible and tested. Building that from scratch would consume most of your build time and none of it would demonstrate anything interesting about your system design.

Component	Where you will use it
`DataGrid`	Applications list, admin user table, transaction ledger — sorting, filtering, pagination included
`Autocomplete`	Skill selection with multi-select chips, backed by your `skills` table
`Stepper`	Milestone progress, multi-step job creation
`Dialog`, `Drawer`	Proposal review, filter panels
`Chip`, `Badge`	Statuses, skill tags, unread counts
`@mui/x-date-pickers`	Interview scheduling, milestone due dates

8.2 The App Router setup gotcha

> MUI uses Emotion for styling, which needs its style cache wired into Next's server rendering. Skip this and you get a flash of unstyled content on every server-rendered page, plus hydration mismatch warnings. This is the single most common MUI + App Router problem.

```
// app/layout.tsx
import { AppRouterCacheProvider } from '@mui/material-nextjs/v15-appRouter';
import { ThemeProvider } from '@mui/material/styles';
import CssBaseline from '@mui/material/CssBaseline';
import theme from '@/theme';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>
        <AppRouterCacheProvider>
          <ThemeProvider theme={theme}>
            <CssBaseline />
            <ReduxProvider>{children}</ReduxProvider>
          </ThemeProvider>
        </AppRouterCacheProvider>
      </body>
    </html>
  );
}
```

8.3 Theming — the difference between "used MUI" and "designed with MUI"

Default MUI is instantly recognisable, and a reviewer who has seen fifty portfolio projects will notice. A custom theme costs an hour and changes the impression substantially.

```
// theme.ts
import { createTheme } from '@mui/material/styles';

const theme = createTheme({
  palette: {
    primary:   { main: '#1d4f8c' },
    secondary: { main: '#0f766e' },
    background:{ default: '#f7f9fb' },
  },
  typography: {
    fontFamily: 'var(--font-inter), system-ui, sans-serif',
    h1: { fontSize: '2rem', fontWeight: 700, letterSpacing: '-0.02em' },
    button: { textTransform: 'none' },   // removes the dated ALL-CAPS default
  },
  shape: { borderRadius: 10 },
  components: {
    MuiButton: {
      defaultProps: { disableElevation: true },
      styleOverrides: { root: { fontWeight: 600 } },
    },
    MuiCard: {
      styleOverrides: { root: { border: '1px solid #e2e8f0', boxShadow: 'none' } },
    },
  },
});
```

Two changes carry most of the effect: `textTransform: 'none'` on buttons, and replacing default shadows with subtle borders. Both move the design away from 2018-era Material.

8.4 Bundle size — the honest weakness

MUI is heavy. Mitigations, in order of impact: import from the package root (`import { Button } from '@mui/material'`) so tree-shaking works — the per-file deep import path is no longer needed and hurts in some bundlers; lazy-load `DataGrid` and date pickers with `next/dynamic`, since they are large and appear on few routes; and keep MUI out of Server Components so it does not inflate the initial payload of public pages.

> **If asked "why MUI over Tailwind or shadcn/ui?":** MUI gives you complex behavioural components — data grid, date picker, autocomplete — that Tailwind does not, because Tailwind is a styling system rather than a component library. For an admin-heavy product that is a real time-saving. The cost is bundle size and a more opinionated look. If the product were marketing-led and design-differentiated, I would choose Tailwind with shadcn/ui instead. Naming the condition under which you would choose differently is what makes this a decision rather than a default.

9. Redux Toolkit & RTK Query

9.1 The distinction that matters most

Most Redux mistakes come from one confusion. There are two kinds of state, and they need different tools.

Client state → Redux slices	Server state → RTK Query
Owned entirely by the browser	Owned by the database; the browser holds a copy
Never stale — it is the truth	Always potentially stale
Synchronous	Asynchronous, can fail, needs retry
Examples: current user, sidebar open, active filters, unsent chat draft, theme mode	Examples: job list, applications, match scores, messages, wallet balance

> **The anti-pattern to avoid:** a `jobsSlice` with `items`, `loading`, `error` and a `fetchJobs` thunk. Repeat that across twelve resources and you have written several hundred lines that reimplement caching, deduplication, refetch-on-focus and invalidation — worse than the version that ships inside the library you already installed.

9.2 RTK Query API definition

```
// store/api/jobsApi.ts
export const jobsApi = createApi({
  reducerPath: 'jobsApi',
  baseQuery: fetchBaseQuery({
    baseUrl: `${process.env.NEXT_PUBLIC_API_URL}/api`,
    credentials: 'include',
    prepareHeaders: (headers, { getState }) => {
      const token = (getState() as RootState).auth.accessToken;
      if (token) headers.set('authorization', `Bearer ${token}`);
      return headers;
    },
  }),
  tagTypes: ['Job', 'Application', 'Proposal'],
  endpoints: (builder) => ({

    getJobs: builder.query<PaginatedJobs, JobFilters>({
      query: (filters) => ({ url: '/jobs', params: filters }),
      providesTags: (result) =>
        result
          ? [...result.data.map(({ id }) => ({ type: 'Job' as const, id })),
             { type: 'Job', id: 'LIST' }]
          : [{ type: 'Job', id: 'LIST' }],
    }),

    getJob: builder.query<Job, string>({
      query: (id) => `/jobs/${id}`,
      providesTags: (_r, _e, id) => [{ type: 'Job', id }],
    }),

    createJob: builder.mutation<Job, CreateJobInput>({
      query: (body) => ({ url: '/jobs', method: 'POST', body }),
      // Creating a job invalidates the list, which refetches automatically
      invalidatesTags: [{ type: 'Job', id: 'LIST' }],
    }),

    updateApplicationStage: builder.mutation<Application, MoveStageInput>({
      query: ({ id, ...body }) => ({ url: `/applications/${id}/stage`,
                                     method: 'PATCH', body }),
      invalidatesTags: (_r, _e, { id }) => [{ type: 'Application', id },
                                            { type: 'Application', id: 'LIST' }],
    }),
  }),
});

export const { useGetJobsQuery, useGetJobQuery, useCreateJobMutation,
               useUpdateApplicationStageMutation } = jobsApi;
```

Consuming it in a component is three lines:

```
function JobList({ filters }: { filters: JobFilters }) {
  const { data, isLoading, error } = useGetJobsQuery(filters);

  if (isLoading) return <Skeleton variant="rectangular" height={400} />;
  if (error)     return <Alert severity="error">Could not load jobs</Alert>;

  return <Stack spacing={2}>{data!.data.map(j => <JobCard key={j.id} job={j} />)}</Stack>;
}
```

No thunk, no reducer, no loading flag in a slice. RTK Query deduplicates concurrent requests for the same key, caches by argument, and refetches on tag invalidation.

9.3 Slices for genuine client state

```
// store/slices/authSlice.ts
const authSlice = createSlice({
  name: 'auth',
  initialState: { user: null, accessToken: null, status: 'idle' } as AuthState,
  reducers: {
    credentialsSet(state, action: PayloadAction<{ user: User; token: string }>) {
      state.user = action.payload.user;      // Immer — safe to "mutate"
      state.accessToken = action.payload.token;
      state.status = 'authenticated';
    },
    loggedOut(state) {
      state.user = null;
      state.accessToken = null;
      state.status = 'idle';
    },
  },
});
```

Redux Toolkit wraps Immer, so writing `state.user = x` produces an immutable update under the hood. Explaining that — rather than believing you are mutating state — is a common interview follow-up.

9.4 Optimistic updates for the pipeline board

Dragging a candidate between stages should feel instant. RTK Query supports this directly, and it is a good thing to demo.

```
updateApplicationStage: builder.mutation<Application, MoveStageInput>({
  query: ({ id, ...body }) => ({ url: `/applications/${id}/stage`,
                                 method: 'PATCH', body }),
  async onQueryStarted({ id, toStageId, jobId }, { dispatch, queryFulfilled }) {
    // Apply the change locally before the server responds
    const patch = dispatch(
      jobsApi.util.updateQueryData('getApplications', { jobId }, (draft) => {
        const app = draft.find(a => a.id === id);
        if (app) app.currentStageId = toStageId;
      })
    );
    try {
      await queryFulfilled;
    } catch {
      patch.undo();   // Server rejected — roll the UI back
    }
  },
}),
```

9.5 Wiring the store into the App Router

```
// store/Provider.tsx
'use client';
import { Provider } from 'react-redux';
import { store } from './store';

export function ReduxProvider({ children }: { children: React.ReactNode }) {
  return <Provider store={store}>{children}</Provider>;
}
```

The `'use client'` directive is mandatory — the Redux Provider uses React context, which Server Components cannot provide. Wrap it once in the root layout.

10. Project Structure — Both Repositories

```
frontend/
├── src/
│   ├── app/
│   │   ├── (public)/                  route group — no auth
│   │   │   ├── page.tsx               landing
│   │   │   ├── jobs/page.tsx          SSR, indexable
│   │   │   └── jobs/[id]/page.tsx     SSR + generateMetadata
│   │   ├── (candidate)/
│   │   │   ├── layout.tsx             'use client' — guarded shell
│   │   │   ├── dashboard/
│   │   │   ├── applications/
│   │   │   └── contracts/[id]/
│   │   ├── (recruiter)/
│   │   │   ├── jobs/[id]/pipeline/    drag-and-drop board
│   │   │   └── proposals/
│   │   ├── (admin)/
│   │   │   ├── disputes/
│   │   │   └── financials/
│   │   └── layout.tsx                 providers mount here
│   ├── components/
│   │   ├── ui/                        thin MUI wrappers
│   │   ├── jobs/  applications/  chat/  milestones/
│   ├── store/
│   │   ├── store.ts
│   │   ├── slices/                    auth · ui · filters
│   │   └── api/                       RTK Query endpoint files
│   ├── hooks/                         useAppDispatch, useAppSelector, useSocket
│   ├── lib/                           socket client, formatters, guards
│   ├── types/                         shared API types
│   └── theme.ts
```

A high-value trick: share types between the repos. Prisma generates types on the backend. Export the ones your API returns from a small `packages/shared-types` workspace, or generate an OpenAPI spec and derive frontend types from it. Either way, renaming a database column produces a frontend compile error rather than an `undefined` at runtime. This is the concrete payoff of TypeScript everywhere, and it is worth demonstrating rather than merely claiming.

11. Testing Strategy

Level	Tools	What to cover — in priority order
Unit	Vitest / Jest	Fee and split calculations, state-transition guards, scoring logic. Pure functions, no database.
Integration	Vitest + Testcontainers or a test Postgres	**Highest value for this project.** The milestone release transaction, the duplicate-application constraint, escrow release. Test against a real database — an in-memory mock cannot test a partial unique index.
API	Supertest	Auth flows, IDOR attempts (candidate B requesting candidate A's application must 403)
Component	React Testing Library	Forms, validation messages, conditional rendering by role
E2E	Playwright	One happy path per flow: post → apply → hire, and post → bid → contract → release

> **If you only write a handful of tests, write these:** the concurrent-payout test that proves two simultaneous withdrawals cannot overdraw a wallet, and the duplicate-application test that proves the constraint holds. Both demonstrate that you understood the hard parts, and both make excellent things to point at in an interview.

12. Deployment

Piece	Where	Notes
Frontend	Vercel	Native Next.js support, previews per pull request
API + Socket.IO	Railway / Render / Fly.io	**Must be a long-running process** — WebSockets rule out serverless
Worker	Same platform, separate process	BullMQ consumer for AI scoring and email
PostgreSQL	Neon / Supabase / RDS	Managed, with point-in-time recovery. You are storing payment records.
Redis	Upstash	Sessions, rate limiting, queue
Files	Cloudflare R2 / S3	Presigned uploads direct from browser

```
# CI pipeline, in order — the middle step is the one people forget
npm ci
npx prisma generate          # regenerate the typed client
npx prisma migrate deploy    # apply pending migrations — never `migrate dev`
npm run build
npm test
```

13. Interview Question Bank

Thirty-eight questions specific to this stack, grouped by area. Answers are written the way you would say them — position first, reasoning second.

13.1 Stack justification

1. Why this stack?

> TypeScript end to end with Prisma means the database schema generates types that reach into React components, so a column rename is a compile error rather than a runtime bug — worth a lot on a forty-one table schema. Postgres because the payment flow needs real transactions and my integrity rules use partial indexes and CHECK constraints. Next.js because job listings need SEO while dashboards need interactivity, and it does both. MUI for component velocity on an admin-heavy product. Redux Toolkit with RTK Query so client and server state stay separated.

2. Why Prisma over TypeORM, Sequelize, or raw SQL?

> Type safety generated from the schema rather than hand-maintained, a migration workflow that produces reviewable SQL files, and relation loading that is readable. The trade-off is that complex queries and features like `SELECT FOR UPDATE` need raw SQL — I use `$queryRaw` where that applies, so I get the ergonomics for the ninety percent and full SQL control for the rest.

3. Isn't an ORM a performance problem?

> It can be if used carelessly — the N+1 pattern is the usual cause, and Prisma makes it easy to write. I avoid it with `include` / `select`, verify by enabling query logging in development, and drop to raw SQL for the analytics-shaped queries where an ORM has nothing to offer. The ORM is a default, not a rule.

4. Why Next.js rather than plain React with Vite?

> SEO. Job listings and company pages must be indexable, and a client-rendered SPA serves Google an empty div. Next also gives file-based routing, image optimisation and API routes if needed. For a purely private dashboard I would have used Vite — the public half is what makes Next the right call.

5. Why MUI rather than Tailwind or shadcn/ui?

> The product is admin-heavy — data grids, date pickers, autocomplete, steppers. MUI ships those as accessible behavioural components; Tailwind is a styling system and would leave me building them. The cost is bundle size and a recognisable default look, which I addressed with a custom theme. For a design-differentiated marketing product I would choose Tailwind with shadcn/ui instead.

6. Why Redux at all? Isn't Context enough?

> Context is a dependency-injection mechanism, not a state manager — any change re-renders every consumer, and there is no devtools timeline or middleware. For auth plus UI plus filters plus cached server data across a large app, Redux Toolkit's devtools and RTK Query's caching are worth the setup. For a small app with two pieces of shared state I would use Context and skip Redux.

13.2 Prisma and the database layer

7. Your schema needs CHECK constraints and partial indexes. Prisma doesn't support those. What did you do?

> Generated empty migrations with `prisma migrate dev --create-only` and hand-wrote the SQL, which Prisma then tracks in migration history like any other. It matters because those rules are load-bearing — the partial unique index is what stops duplicate live applications while still allowing reapplication after withdrawal. If I had relied only on the Prisma schema language, none of those constraints would exist in the database.

8. Walk me through your milestone release transaction.

> It runs inside `prisma.$transaction`. First a raw `SELECT ... FOR UPDATE` locks the wallet row, because Prisma has no API for row locking and the lock is what actually prevents double-spend. Then a guarded `updateMany` that only matches if the milestone is still `SUBMITTED` — if another request already approved it, zero rows match and I abort rather than pay twice. Then escrow release, fee record, wallet credit and ledger entry, all in the same transaction so a crash rolls everything back. Money is never created or destroyed by a partial failure.

9. Why raw SQL for the lock? Can't Prisma do it?

> Prisma has no `FOR UPDATE` API. It is a known limitation. I use `tx.$queryRaw` with the transaction client — importantly not the global `prisma` client, which would open a separate connection outside the transaction and silently break atomicity.

10. What is the N+1 problem and how does Prisma expose you to it?

Fetching a list, then querying once per item in a loop — fifty applications become fifty-one queries. Prisma invites it because `findUnique` inside a loop reads perfectly naturally. The fix is `include` or `select` to fetch relations in one query. I catch it by enabling query logging in development and watching the count per request.

11. How do you handle money in JavaScript?

Never as a `Number`. The column is `DECIMAL(12,2)` and Prisma returns a `Decimal` object backed by decimal.js, so arithmetic uses `.mul()` and `.sub()`. At the API boundary I serialise with `.toFixed(2)` as a string, and the frontend treats money as a string until display. Floating point cannot represent 0.1 exactly, and those errors accumulate across a ledger.

12. What is the difference between migrate dev and migrate deploy?

`migrate dev` generates new migrations from schema changes, applies them, and can prompt or reset — development only. `migrate deploy` only applies already-generated pending migrations, never prompts, never resets. Production and CI use deploy. Running `migrate dev` against production risks data loss.

13. How do you avoid exhausting database connections?

A singleton `PrismaClient` cached on `globalThis` in development, because hot reload otherwise creates a new pool per reload until Postgres refuses connections. In production I run the API as a long-lived process rather than serverless, which keeps one pool. If I did go serverless I would need PgBouncer in transaction mode or Prisma Accelerate.

14. How does Prisma protect against SQL injection?

All generated queries are parameterised. For raw SQL I use the tagged-template form, `$queryRaw`... ${value}``, which also parameterises. The dangerous escape hatch is `$queryRawUnsafe` with string concatenation, which I do not use.

15. How do you handle a unique constraint violation cleanly?

Catch Prisma's `P2002` error code in the global error middleware and map it to a 409 with a readable message. The database enforces the rule; the middleware translates it. That means the rule lives in exactly one place and I do not need a pre-check query that would race anyway.

13.3 Node, Express and API design

16. How is your backend structured?

Feature modules, each with controller, service and repository layers. Controllers handle only HTTP, services hold business rules and transaction boundaries, repositories hold Prisma calls. The test for whether the layering is real is whether a queue worker could call the service without touching the controller — if the service reads `req.user`, the layers are decorative.

17. Node is single-threaded. Doesn't that limit you?

For an I/O-bound API it is fine — the event loop handles concurrent requests while waiting on the database. The risk is CPU-bound work blocking the loop, which is exactly why AI match scoring runs in a separate worker process consuming a BullMQ queue rather than inline in the request. For scaling across cores I would run multiple instances behind a load balancer rather than use `cluster` directly.

18. Where do you validate input?

Zod schemas at the route boundary, with types inferred from the schema so there is one definition. It duplicates some database CHECK constraints deliberately — the constraint is the guarantee, the Zod schema is the good error message. Users should see "salary range is backwards", not a raw constraint violation.

19. How do you prevent one user reading another's data?

Ownership checks in the service layer, not just role checks in middleware. Role middleware answers "is this a recruiter"; it does not answer "is this *their* job". Skipping the second check is an IDOR vulnerability, and it is the most common real bug in marketplace apps. Chat access derives from membership in `chat_participants` rather than from the client claiming a room ID.

20. How do you make the payment webhook idempotent?

A unique constraint on the gateway's transaction reference. If the provider delivers the same webhook twice, the second insert fails at the database level regardless of the handler code. The handler catches the duplicate-key error and returns 200, because the work was already done.

13.4 Next.js and rendering

21. What is the difference between a Server and a Client Component?

Server Components run only on the server, ship no JavaScript, and can fetch data directly — but cannot use hooks, state or event handlers. Client Components are marked `'use client'`, ship to the browser, and can do all of that. In my app the public job pages are Server Components for SEO and payload size, and every dashboard is a Client Component subtree because Redux and MUI both need client context.

22. How did you decide the rendering strategy per page?

By whether the page needs to be indexed. Job listings and detail pages are server-rendered with revalidation, because a job board that Google cannot crawl has a product problem. Dashboards, pipeline and chat are client-rendered — private, no SEO value, and highly interactive. The landing and company pages are static.

23. What is hydration and what breaks it?

React attaching event handlers to server-rendered HTML in the browser. It breaks when the server and client render different markup — commonly from `Date.now()`, random values, or reading `window` during render. With MUI it breaks specifically if the Emotion style cache is not wired into the App Router, which is why `AppRouterCacheProvider` is in my root layout.

24. Why can't Socket.IO run on Vercel?

Serverless functions are short-lived and cannot hold a persistent connection. So the API and Socket.IO run as a long-lived process on Railway while the frontend stays on Vercel. If I scaled to multiple API instances I would add the Redis adapter, otherwise two users in the same chat could land on different servers and never see each other's messages.

13.5 Redux and state management

25. What goes in Redux and what does not?

Redux slices hold client state — the authenticated user, UI toggles, active filters, unsent chat drafts. Server data goes in RTK Query. The distinction is ownership: client state is the truth and is never stale; server state is a cached copy of something the database owns and always might be. Putting server data in slices means hand-writing caching and invalidation, badly.

26. What does RTK Query actually give you?

Caching keyed by query argument, request deduplication, automatic refetch on tag invalidation, loading and error states, and generated hooks. Creating a job invalidates the job list tag and the list refetches with no manual wiring. Replacing that with thunks would be several hundred lines of worse code.

27. Explain Immer's role in Redux Toolkit.

RTK wraps reducers in Immer, so writing `state.user = action.payload` looks like mutation but produces a new immutable state via a proxy that records the changes. It removes the spread-operator noise that made classic Redux verbose, without giving up immutability.

28. How did you make the pipeline board feel instant?

Optimistic updates. RTK Query's `onQueryStarted` patches the cache locally the moment a card is dropped, then awaits the server. If the request fails, `patch.undo()` rolls the UI back. The user sees no latency on the happy path and a correct rollback on the unhappy one.

29. Why not TanStack Query instead of RTK Query?

TanStack Query is arguably the better server-state library in isolation. RTK Query wins here because I already need Redux for client state, and RTK Query ships inside it — one store, one set of devtools, one dependency. If I did not need Redux at all, I would use TanStack Query and skip Redux entirely.

13.6 TypeScript

30. What does TypeScript actually buy you on this project?

The concrete payoff is that Prisma generates types from the schema, so renaming a database column produces a compile error in the React component that used it. On forty-one tables that catches a whole class of bug that would otherwise reach runtime. It also makes the API contract between the two repos checkable rather than assumed.

31. Difference between type and interface?

`interface` supports declaration merging and is idiomatic for object shapes that may be extended. `type` handles unions, intersections, mapped and conditional types. Practically I use `interface` for object contracts and `type` for everything else, especially the union types that model my status enums.

32. What is a discriminated union and where did you use one?

A union of object types sharing a literal field the compiler can narrow on. My jobs are the natural case — a `FULL_TIME` job has a salary range and a `FREELANCE` job has a budget, so discriminating on `jobType` lets TypeScript know which fields exist in each branch. It mirrors the CHECK constraint in the database at the type level.

33. How do you avoid `any`?

`strict` mode on, `unknown` instead of `any` at untyped boundaries, then narrow with a Zod parse. Zod is doing double duty — runtime validation and type inference — so external data enters the system typed and validated at the same point.

13.7 Performance and scale

34. A job gets ten thousand applicants. What breaks?

Three things. The pipeline board if it loads all applications — so it paginates on an indexed `(job_id, status)`. Match scoring if it runs inline — so it is queued and processed by a worker. And the MUI DataGrid if it renders ten thousand rows — so it uses server-side pagination rather than client-side.

35. How would you speed up the job search page?

First measure with `EXPLAIN ANALYZE` rather than guessing. Then: the composite index on `(status, job_type, created_at DESC)` for the default browse, cursor pagination instead of `OFFSET` which degrades on deep pages, Next's revalidation so the SSR result is cached at the edge, and Postgres full-text search on title and description before reaching for Elasticsearch.

36. How do you reduce the frontend bundle?

`next/dynamic` for the DataGrid and date pickers, which are large and used on few routes. Keep MUI out of Server Components so public pages stay light. Route-level code splitting is automatic in the App Router. And `@next/bundle-analyzer` to find what actually dominates rather than guessing.

37. What would you cache in Redis?

Sessions and refresh-token denylists, rate-limit counters, and the BullMQ queue. I would also cache the AI-ranked shortlist per job, since it changes only when scores are recomputed but is read on every page load. I would not cache anything in the payment path — stale money data is worse than slow money data.

38. What was the hardest bug or lesson on this project?

Realising a `CHECK (balance >= 0)` does not prevent double-spending. Two concurrent payouts can both read a balance of five hundred, both pass the check, and both commit — the constraint is never violated at any instant either transaction observed. The fix was `SELECT ... FOR UPDATE` to serialise them. It taught me that constraints protect against bad data, not against bad concurrency, and those are different problems.

14. Trade-offs & What You Would Change

Every interview ends here. The correct shape is: the limitation, why you accepted it, and what would change your mind.

Limitation	Why accepted	What would change it
Express gives no structure — layering is maintained by discipline	Smaller mental model; full control over wiring	A growing team. NestJS enforces module boundaries rather than trusting convention
Prisma cannot express CHECK constraints or partial indexes	Hand-written migration SQL covers it, and Prisma tracks those files	If the count grew large, Drizzle or Kysely stay closer to SQL
MUI bundle size	Enormous component velocity on an admin-heavy product	A public-facing, design-led product where first paint is the metric
REST rather than GraphQL	Simpler caching, simpler auth, and RTK Query handles REST well	Many clients with divergent data needs, or heavy over-fetching
Two repositories with types shared manually	Independent deploys; simpler CI	A monorepo with a shared types package, or generating clients from OpenAPI
Socket.IO in the API process	Shared auth and one deployment	Chat volume warranting a dedicated service with the Redis adapter
Postgres full-text search rather than Elasticsearch	Adequate to roughly a hundred thousand rows; one less system	Fuzzy matching, relevance tuning, or search latency past ~100ms

> **A closing answer that works:** "The choice I would most want to revisit is Express over NestJS. My layering is real, but it is maintained by convention — nothing stops a future contributor calling Prisma from a controller. NestJS would have made that structurally difficult rather than merely discouraged. At solo scale I preferred the smaller mental model, and I think that was right, but it is the decision most dependent on team size."

15. Build Order — What to Do First

The most common way this project fails is building all forty-one tables badly instead of one flow well. A working freelance flow with real escrow demos far better than two half-finished halves.

Phase	Build	Why this order
1	Prisma schema, migrations, the hand-written constraint migration, seed data	Everything depends on it; getting constraints in from day one avoids retrofitting against dirty data
2	Auth: register, login, refresh, role middleware, guarded layouts	Every other feature needs a logged-in user
3	Jobs CRUD + public listing with SSR	First visible, demoable slice; proves the Next rendering split works
4	Proposals → contract → milestones → escrow → release	**The centrepiece.** Transactions, locking, ledger — the technically hardest and most impressive part
5	Chat with Socket.IO, scoped to contracts	Needed for delivery, and enables dispute evidence
6	Full-time flow: applications, stages, pipeline board, offers	Reuses jobs and auth; the drag-and-drop board is the best UI demo
7	AI matching in a worker, writing `match_scores`	Async by design; can be a heuristic scorer first and improved later
8	Admin: disputes, moderation, financials	Depends on everything above existing
9	Reviews, notifications, polish, tests	Value-adding but not load-bearing

The single highest-value thing you can build: phase 4, end to end, with the concurrency test that proves two simultaneous payouts cannot overdraw a wallet. That one feature demonstrates transactions, row locking, decimal arithmetic, state machines, idempotency and integration testing — six things that are hard to fake and that most portfolio projects do not have. If time runs short, cut phases 7 through 9 before cutting anything from phase 4.

Final note on interviews. You will not be asked thirty-eight questions. You will be asked four or five, and the difference between a good and a poor answer is whether you supply the reasoning unprompted. "I used Prisma" is a fact. "I used Prisma for type-safe queries generated from the schema, and dropped to raw SQL for row locking because Prisma has no FOR UPDATE API" is an answer that demonstrates you hit the limits of your tools and understood them. Aim for the second form every time.